

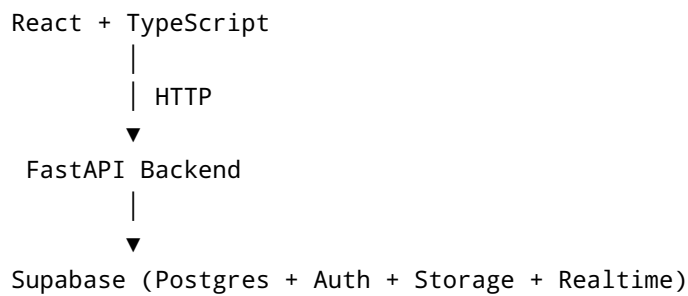
3. Technical Architecture

This chapter describes the architecture of Peregri and the reasoning behind every major technical decision.

The goal is not only to document how the project currently works, but also *why* it was designed this way. Future contributors and AI agents should understand the reasoning before modifying the codebase.

Architecture Overview

Peregri follows a classic modern web architecture.



The frontend is intentionally separated from the backend.

Supabase is **not** treated as the application's backend.

Instead:

- Supabase manages infrastructure.
- FastAPI manages business logic.
- React manages presentation.

Keeping these responsibilities separate makes future migration significantly easier.

Why FastAPI?

Many Supabase projects call the database directly from the frontend.

This project intentionally does **not**.

Reasons:

- Business logic belongs on the server.
- Easier validation.
- Better security.
- Easier future integrations.
- Cleaner architecture.
- Easier payment implementation.
- Easier admin tools.
- Easier scaling.

The frontend should never become responsible for complex business rules.

Technology Stack

Frontend

- React
- TypeScript
- Vite
- TanStack Router
- Tailwind CSS v4
- shadcn/ui

React is responsible only for rendering UI.

Business logic should remain lightweight.

Whenever possible, complicated logic belongs inside reusable hooks or backend endpoints.

Backend

FastAPI

Reasons:

- Fast
- Excellent typing
- Easy validation
- Automatic documentation
- Async support
- Simple deployment

The backend is the source of truth for marketplace operations.

Database

Supabase PostgreSQL.

Reasons:

- Managed PostgreSQL.
- Authentication.
- Storage.
- Realtime.
- RLS support.
- Excellent ecosystem.

Although Supabase provides many backend capabilities, the project intentionally avoids pushing business logic into SQL whenever a Python implementation is cleaner.

Frontend Architecture

The frontend is organized by feature.

Not by file type.

Good:

```
Browse/  
  
CreateListing/  
  
Messages/  
  
Profile/
```

Avoid:

```
buttons/  
  
inputs/  
  
helpers/  
  
screens/
```

misc/

Features should remain relatively independent.

Component Philosophy

Components should remain small.

Ideal responsibilities:

One component.

One purpose.

Examples:

ListingCard

UserAvatar

ConversationList

MessageBubble

PriceTag

Instead of:

Huge 800-line components doing everything.

Smart vs Dumb Components

Presentation components:

- receive props
- render UI

Logic components:

- fetch data
- perform mutations

- manage state

Mixing both is acceptable for very small pages but should be avoided as the project grows.

Routing

TanStack Router is used.

Reasons:

- Type safety.
- Nested layouts.
- Search parameter validation.
- Better scalability.

Navigation should remain declarative.

Avoid manipulating `window.location` unless absolutely necessary.

Authentication

Supabase Auth manages authentication.

The frontend stores:

- Session
- Current user
- Basic profile

Additional profile information lives inside the project's own `profiles` table.

This separation is intentional.

Authentication belongs to Supabase.

Application data belongs to the application.

User Model

Every authenticated user has:

Supabase Auth user



Profile row

The profile contains information used throughout the marketplace.

Examples:

display_name

avatar

bio

country

city

Future additions:

verification status

review count

rating

completed deliveries

response rate

Why Profiles Exist

Supabase Auth users cannot be freely extended.

Creating a separate profile table provides flexibility.

The application should never rely on auth metadata for business information.

Backend Philosophy

The backend exposes clean REST endpoints.

Examples:

```
GET /listings  
  
POST /listings  
  
GET /messages  
  
POST /threads
```

The frontend should rarely communicate with Supabase directly except for authentication and carefully selected realtime subscriptions.

Database Philosophy

This project strongly prefers explicit queries over implicit magic.

Example:

Instead of complex automatic joins inside PostgREST,

perform explicit queries when appropriate.

This makes behavior easier to understand and debug.

Important Lesson Learned

Originally the backend attempted to load listings using:

```
listings  
  →  
profiles(...)
```

This failed because there was **no foreign key relationship between** `listings.owner_id` **and** `profiles.id`.

The database was intentionally designed differently.

Instead of forcing an artificial relationship,

the backend now performs two separate operations:

1. Load listings.
2. Load the required owner profiles.
3. Merge them in Python.

Although this requires one additional query, it has several advantages:

- explicit
- predictable
- independent of PostgREST relationship detection
- easier to debug
- works regardless of schema cache
- keeps application logic inside the backend

Future contributors should preserve this philosophy.

Auth vs Profiles

One important architectural decision:

```
auth.users
```

is **not**

```
profiles
```

The authentication system owns authentication.

The application owns profile information.

These are different concepts.

Do not merge them.

Realtime Philosophy

Realtime should only be used where users genuinely expect immediate updates.

Examples:

Messages.

Unread counts.

Typing indicators (future).

Notifications.

Realtime should **not** be used simply because it exists.

Polling or normal requests are often simpler.

Unread Counter

One important design decision concerns unread messages.

An early implementation attempted to increment a local counter whenever a realtime event arrived.

This appears efficient but introduces synchronization problems.

Examples:

Multiple browser tabs.

Offline devices.

Reconnects.

Missed websocket events.

Instead, unread count should always be derived from the database whenever correctness matters.

Realtime events should trigger refreshes rather than becoming the source of truth.

Correctness is more important than saving one database query.

Type Safety

The project heavily values type safety.

Frontend:

TypeScript.

Backend:

Python type hints.

Supabase:

Generated database types.

Avoid:

any

unless absolutely necessary.

When types become difficult,

improve the types rather than disabling them.

Constants

Database table names and field names are centralized.

Example:

`DB.TABLES.PROFILES`

`DB.FIELDS.PROFILES.DISPLAY_NAME`

Reasons:

Avoid typos.

Easy refactoring.

Autocomplete.

Consistency.

When using Supabase generated types, remember these constants must remain literal (`as const`) so TypeScript can infer table names correctly.

AI Development Rules

When modifying architecture:

Do **not** rewrite existing systems unless explicitly requested.

Prefer extending existing code.

Preserve naming.

Preserve folder organization.

Avoid introducing additional state management libraries.

Avoid introducing unnecessary abstractions.

Avoid large utility folders.

Avoid premature optimization.

When choosing between:

"clever"

and

"obvious"

prefer obvious.

The codebase should remain understandable six months from now, even if that means writing a few extra lines today.

General Engineering Philosophy

The project intentionally optimizes for:

Readability.

Predictability.

Maintainability.

Consistency.

Not cleverness.

Every future contributor should aim to leave the codebase slightly cleaner than they found it.